Ch 10.3: Convolutional Neural Nets

Lecture 32 - CMSE 381

Prof. Elizabeth Munch

Michigan State University
::
Dept of Computational Mathematics, Science & Engineering

Weds, Dec 6, 2022

Announcements

Last time:

- Multilayer
- pyTorch

This lecture:

- CNNs

Announcements:

- Project due Friday

OH: 11-12 Th Dr Munch

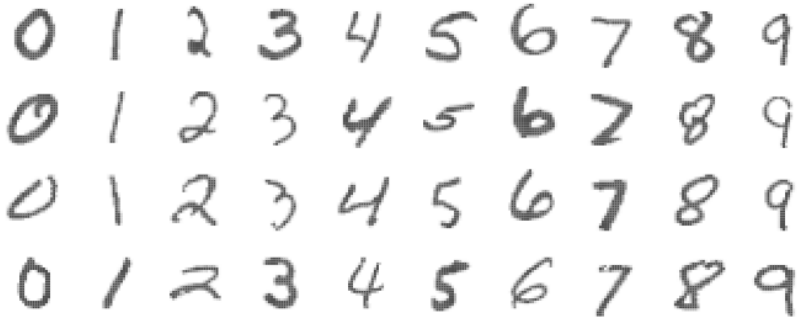
Fr: 11:30-1 Rachel

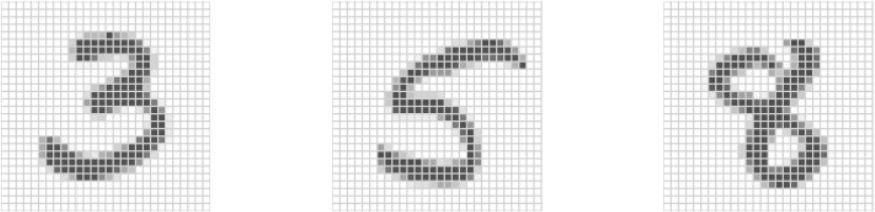
~~Z~~ 3-4:20 OH

Section 1

Last time: Neural Nets

MNIST





Goal: Classify picture by what digit it is

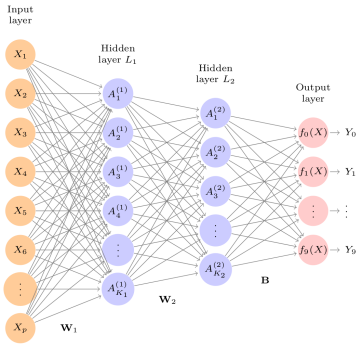
Pic : 28x28 image greyscale [0, 255]

→ flattened images

784 length input vector

Neural network architecture for MNIST

p=784



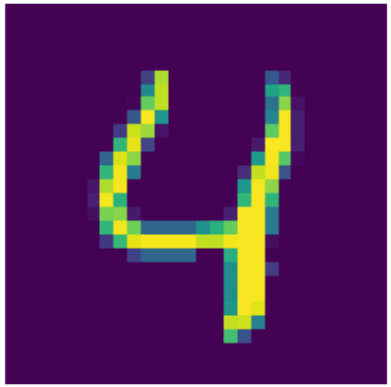
$$
\begin{aligned}
A_k^{(1)} &= h_k^{(1)}(X) \\
&= g(w_{k0}^{(1)} + \textstyle\sum_{j=1}^{p} w_{kj}^{(1)} X_j)
\end{aligned}
$$

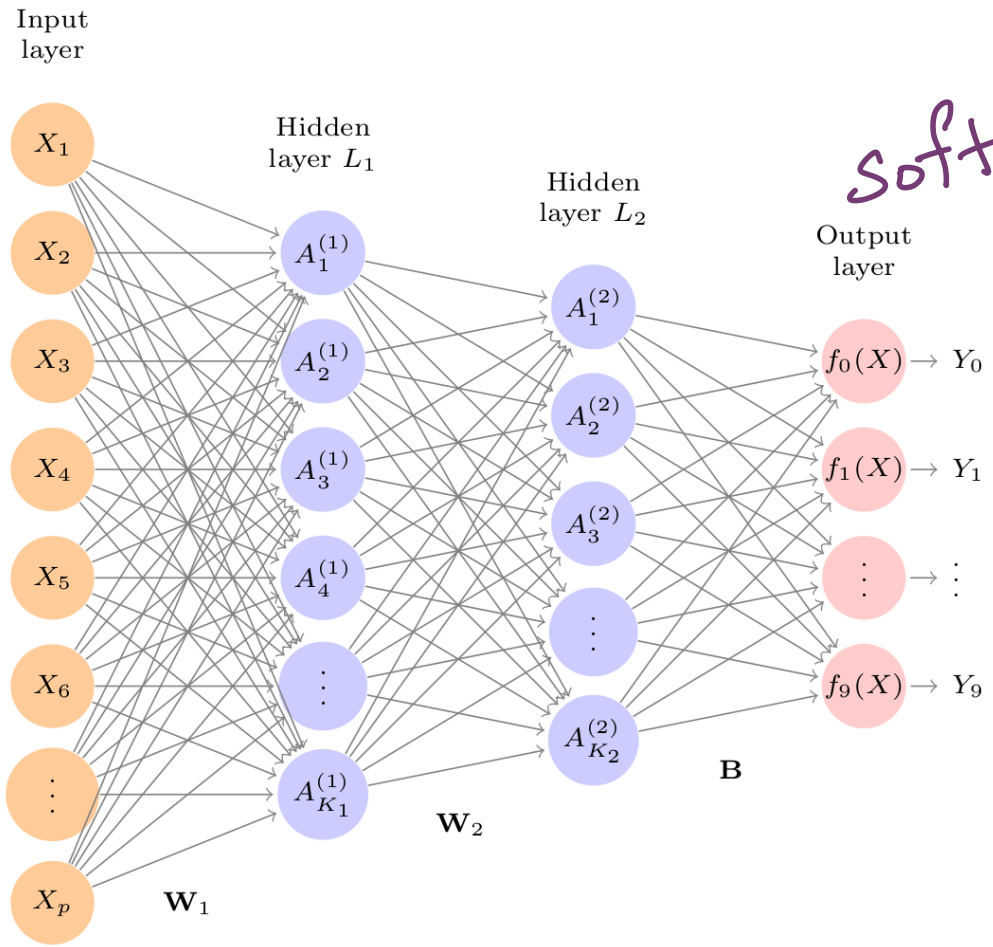
$$
\begin{aligned}
A_\ell^{(2)} &= h_\ell^{(2)}(X) \\
&= g(w_{\ell 0}^{(2)} + \textstyle\sum_{k=1}^{K_1} w_{\ell k}^{(2)} A_k^{(1)})
\end{aligned}
$$

$$
\begin{aligned}
Z_m &= \beta_{m0} + \textstyle\sum_{\ell=1}^{K_2} \beta_{m\ell} h_\ell^{(2)}(X) \\
&= \beta_{m0} + \textstyle\sum_{\ell=1}^{K_2} \beta_{m\ell} A_\ell^{(2)},
\end{aligned}
$$

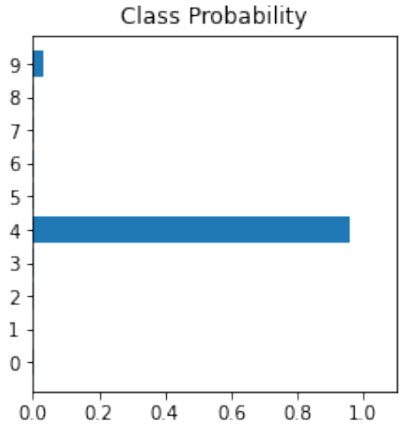
$$
f_m(X) = \Pr(Y = m|X) = \frac{e^{Z_m}}{\sum_{\ell=0}^{9} e^{Z_\ell}},
$$

MNIST learning





softmax



Section 2

Convolutional Neural Network

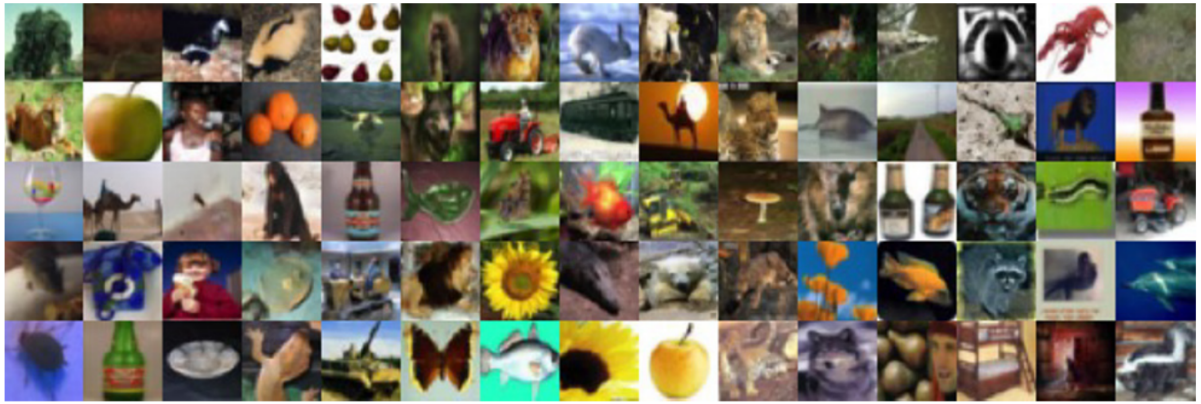
CNN

Last time: Flattening the image

$$\begin{pmatrix} 1 & 1 & 0 \\ 4 & 2 & 1 \\ 0 & 2 & 1 \end{pmatrix} \longrightarrow \begin{pmatrix} 1 \\ 1 \\ 0 \\ 4 \\ 2 \\ 1 \\ 0 \\ 2 \\ 1 \end{pmatrix}$$

Loses a lot of the original structure

Example data set: CIFAR100 Data



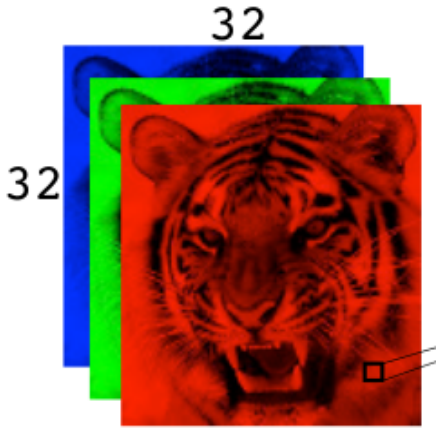
- 60,000 images: 50K training, 10K test
- Labels with 20 super classes (e.g. aquatic mammals)
- 5 classes per super class (beaver, dolphin, otter, seal, whale)
- Images are 32x32

These are in color

Image channel data

Each pixel has 3 values:

R G B (Red green, blue)



Organized in a feature map

3 dimensional array

First two dimensions (32x32) are spatial

Third is channel axis rep'ing the three colors

(32x32x3)

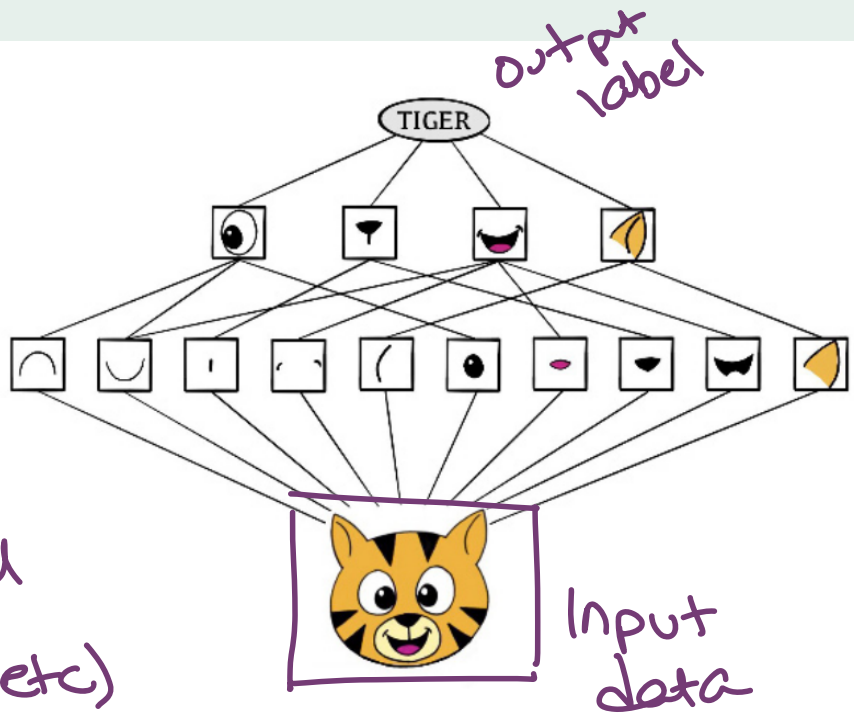
CNNs

Identify low level features on the image such as small edges, colorpatches, etc

Put together into higher-level features (i.e. eyes, ears, etc)

Use two types of layers

convolution layers + pooling layers



Convolution layer

Convolution Filter

Original Image:

$$\begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \\ j & k & l \end{bmatrix}$$

4x3

Convolution filter:

$$\begin{bmatrix} \alpha & \beta \\ \gamma & \delta \end{bmatrix}$$

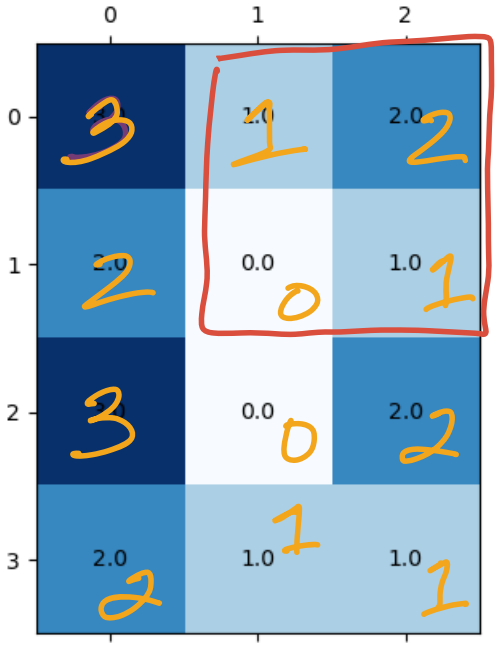
2x2

Convolved Image

$$\begin{bmatrix} a\alpha + b\beta + d\gamma + e\delta & b\alpha + c\beta + e\gamma + f\delta \\ d\alpha + e\beta + g\gamma + h\delta & e\alpha + f\beta + h\gamma + i\delta \\ g\alpha + h\beta + j\gamma + k\delta & h\alpha + i\beta + k\gamma + l\delta \end{bmatrix}$$

3x2

Convolution Filter Example



Filter:

$$\begin{bmatrix} 0 & 1 \\ 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 3\cdot 0 + 1\cdot 1 + 2\cdot 0 + 0\cdot 1 & 1\cdot 0 + 2\cdot 1 + 0\cdot 0 + 1\cdot 1 \\ & \end{bmatrix}$$

$$\begin{bmatrix} 1 & 3 \\ 0 & 3 \\ 1 & 3 \end{bmatrix}$$

Same example, different filter

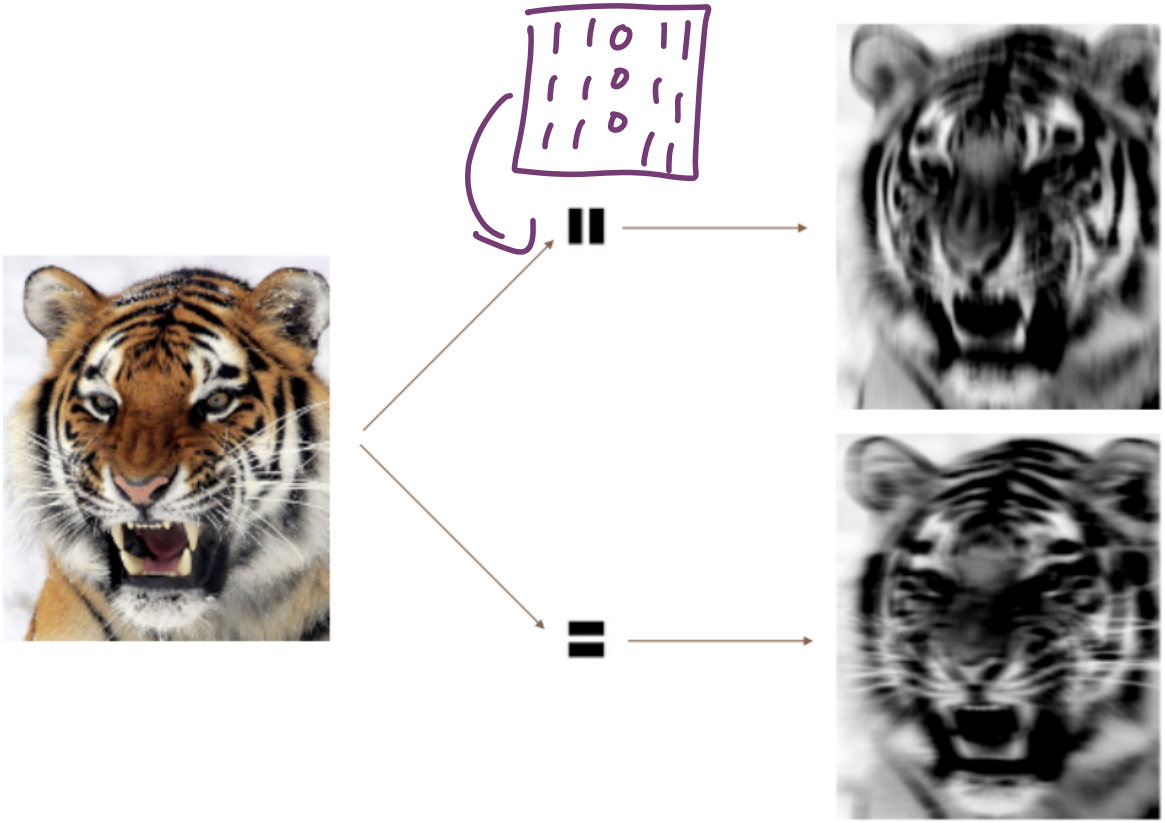
What is the convolved image?

$$\begin{bmatrix} 4 & 3 \\ 2 & 1 \\ 3 & 2 \end{bmatrix}$$

Filter:

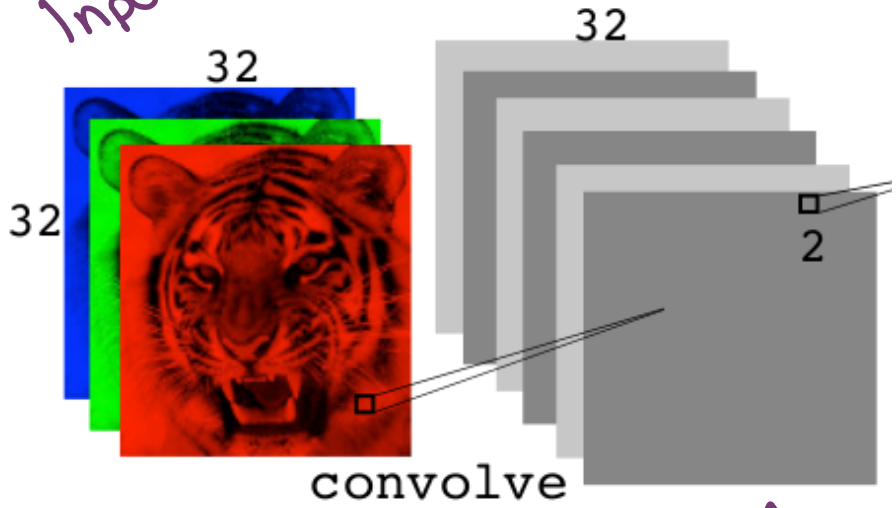
$$\begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix}$$

Convolution filter: Bigger example



Convolution layer

Input data



- Image has 3 channels RGB
- Conv. filters 3 channels (RGB) each of dim 3x3
- Results are summed to form 2D image output
- K diff conv. filters
  - ↳ K diff matrix outputs (treat like channels)

Padding added means output matrices same size as input

More notes on convolution

Idea: for each conv. filter ($n \times n$) matrix
learn the weights (entries)

Version for regular neural nets
one hidden unit per entry in filter matrix

Apply some activation function (e.g. ReLU)
↳ sometimes shown as separate layer

Pooling layers

Goal is:

Condense matrix down

Smaller size

$$\text{Max pool} \quad \begin{bmatrix} 1 & 2 & 5 & 3 \\ 3 & 0 & 1 & 2 \\ 2 & 1 & 3 & 4 \\ 1 & 1 & 2 & 0 \end{bmatrix} \rightarrow \begin{bmatrix} 3 & 5 \\ 2 & 4 \end{bmatrix}$$

Lots of types

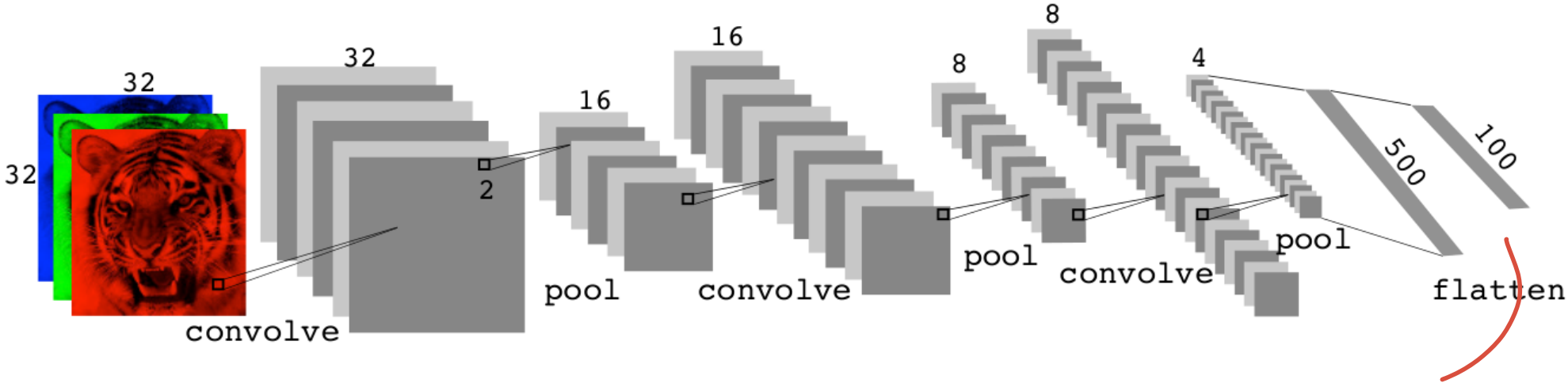
Max pool:

each 2x2 non-overlapping block of pixels

keep highest entry

Idea: Location invariance: high value anywhere results in high value in pooled matrix

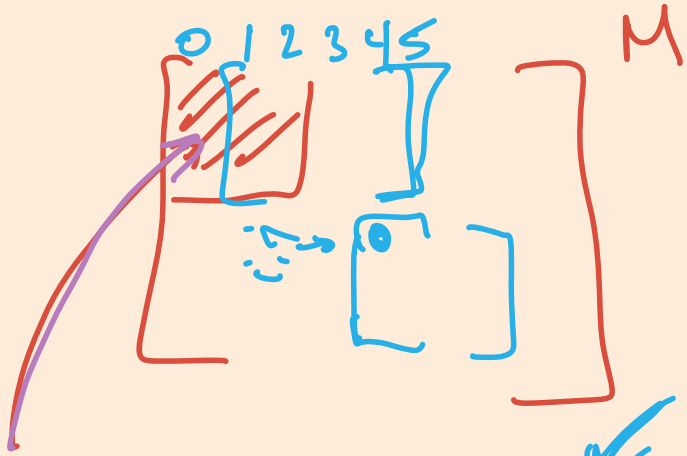
Putting it together to make a CNN



y
Output
label

https://poloclub.github.io/cnn-explainer/

Coding



$M[:5, :5]$

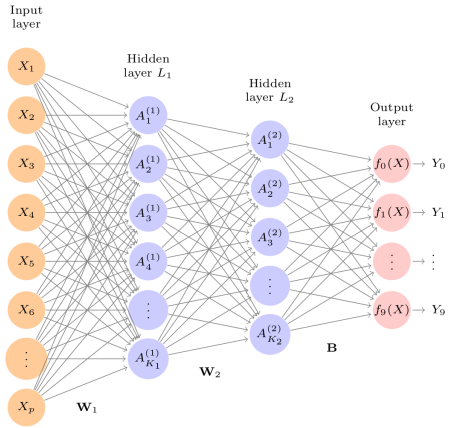
$(0,1)$

$M[:5, 1:6]$

$M[i:i+5, j:j+5]$

TL;DR

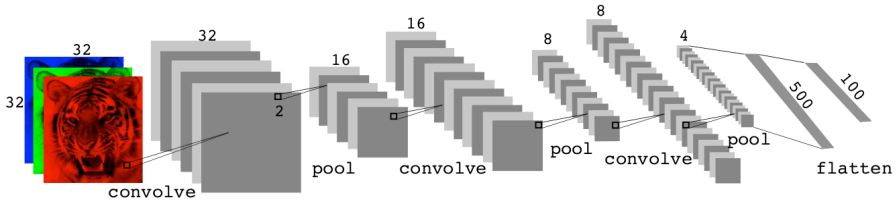
Feed Forward Neural Net



$$A_k = h_k(X) = g(w_{k0} + \sum_{j=1}^{p} w_{kj} X_j),$$

- Combines input data using learned weights
- Linear combo of those to get output
- Sometimes softmax to get probability of classification

CNN



- Specialized NN
- Gets next layer via
  - Convolution layer
  - Pooling Layer
  - Fully connected layer

Next time

Lec #	Date			Reading	Homeworks
	Wed	Nov 29	**Midterm #3**		
32	Fri	Dec 1	Multi Layer NN	10.2	
33	Mon	Dec 4	CNN	10.3	
34	Wed	Dec 6	Unsupervised Learning & Clustering	12.1, 12.4	
35	Fri	Dec 8	Virtual: Project office hours		Project due